Desicion Hub

- トレードオフ分析
- リスク評価
- 意思決定記録
- 選択肢生成
- 撤退基準設計
- 評価基準設計
- 優先順位決定

選択肢生成

選択肢生成

選択肢生成とは、目的や問題に対して取りうる行動案・方針案・戦略案を、比較可能な形で列挙することである。

良い意思決定は、良い比較対象があって初めて成り立つ。
したがって、このノートの役割は「すぐ決めること」ではなく、「決めるに値する案を十分に揃えること」にある。

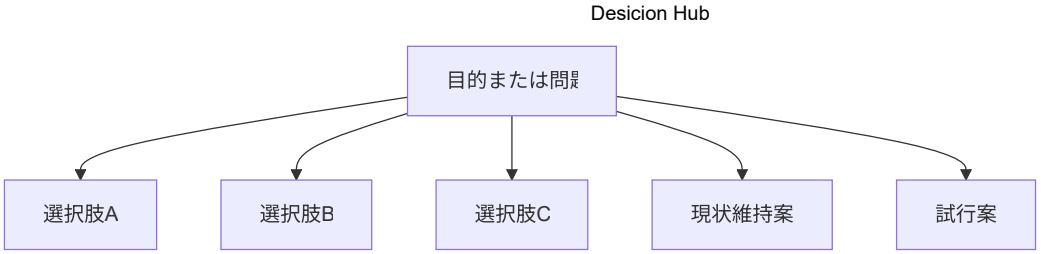
役割

- 早すぎる一本化を防ぐ
- 現状維持も含めて比較対象を作る
- 実行案の幅を確保する
- ハイブリッド案や段階案を見つける
- decision 全体の質を底上げする

選択肢の典型類型

- 現状維持
- 漸進改善
- 部分転換
- 全面転換
- 試行導入
- 一時停止
- 外部委託
- 内製化
- 縮小運用
- 撤退
- ハイブリッド案

基本構造



テンプレート

- 目的:
- 制約:
- 選択肢A:
- 選択肢B:
- 選択肢C:
- 現状維持案:
- 試行案:
- ハイブリッド案:
- 比較のために不足している案:
- 生成時点での直感:

良い選択肢の条件

- 実際に比較可能である
- 互いの差が明確である
- 同じ目的に向いている
- 実行条件が書ける
- 現実案と理想案が混ざっていない

注意点

- 実質一択なのに比較しているふりをしない
- 現状維持を無意識に除外しない
- 逆に、非現実案だけで埋めない
- 案の違いが基準の違いになっていないか確認する

トレードオフ分析

トレードオフ分析

トレードオフ分析とは、ある選択肢を採ることで得られる価値と、同時に失われる価値を明示することである。

現実の意思決定では、全てを最大化できることは稀である。
したがって、何を優先し、何を犠牲にし、どの損失が許容可能で、どの損失が許容不可能かを整理する必要がある。

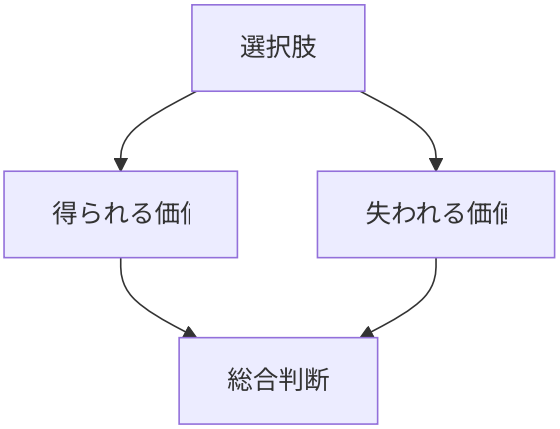
役割

- 見えにくい損失を可視化する
- 得失の交換関係を明確にする
- 不可逆な損失を見落とさない
- 短期利益と長期損失のズレを明示する
- 意思決定後の後悔ポイントを先に見る

何を見るか

- 何を得るか
- 何を失うか
- どの損失は回復可能か
- どの損失は不可逆か
- 時間差で現れる損益は何か
- 誰にとっての得失か

基本構造



テンプレート

- 選択肢:
- 得られるもの:
- 失うもの:
- 短期メリット:
- 長期メリット:
- 短期コスト:
- 長期コスト:
- 可逆な損失:
- 不可逆な損失:
- 利害関係者別の得失:
- 総合判断:

典型的な論点

- 速度 vs 精度
- 効果 vs コスト
- 拡張性 vs 単純性
- 自由度 vs 統制
- 内製化 vs 外部依存
- 短期成果 vs 長期基盤整備

注意点

- 得るものだけを書かない
- 「失うものが少ない」ことと「良い案」を混同しない
- 誰の得か、誰の損かを曖昧にしない
- 不可逆な損失は特に重く扱う

評価基準設計

評価基準設計

評価基準設計とは、複数の選択肢を何によって評価するかを明示することである。

判断の質は、選択肢の質だけでなく、基準の質によって大きく左右される。
基準が曖昧だと、比較は形式だけになり、最終的には雰囲気・好み・惰性で決まりやすい。

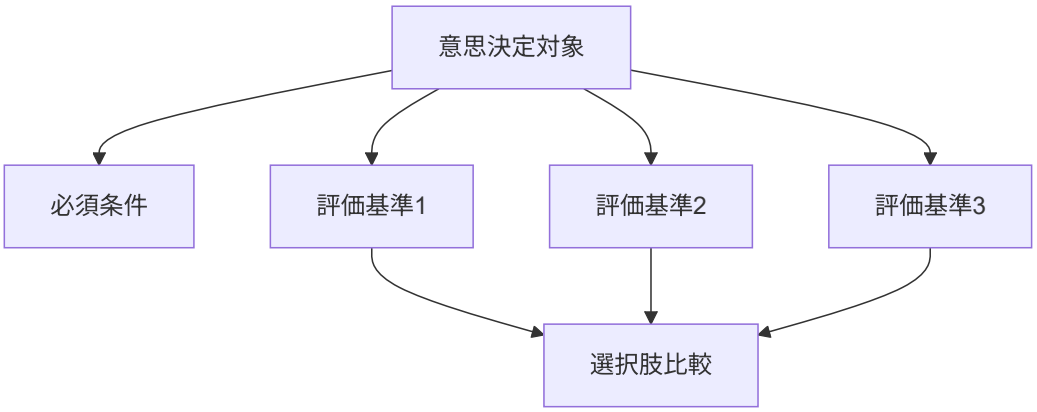
役割

- 比較の軸を揃える
- 判断の恣意性を減らす
- 必須条件と加点条件を分ける
- 短期最適と長期最適を分ける
- 利害関係者間の認識差を明らかにする

典型基準

- 効果
- 実現可能性
- コスト
- 所要時間
- リスク
- 可逆性
- 拡張性
- 学習価値
- 利害関係者受容性
- 法制度適合性
- ブランド整合性
- 維持運用負荷

基本構造



テンプレート

- 意思決定対象:
- 必須条件:
- 禁止条件:
- 評価基準1:
- 評価基準2:
- 評価基準3:
- 重みづけ:
- 判定方法:
- 判断主体:
- 評価の留保点:

良い基準の条件

- 比較に使えるほど具体的である
- 重複が少ない
- 必須条件と加点条件が分かれている
- 短期と長期を混同しない
- 価値判断が隠れていない

注意点

- 効果だけで決めない
- 使いやすさと低コストを同一視しない

- 本当は重要な基準を暗黙化しない
- 重みづけを後出しにしない

リスク評価

リスク評価

リスク評価とは、選択肢ごとに、どのような失敗が起こりうるか、その発生確率・影響度・検知可能性・回復可能性を整理することである。

意思決定では、期待される利益だけでなく、最悪ケース、崩れ方、復旧可能性を考える必要がある。

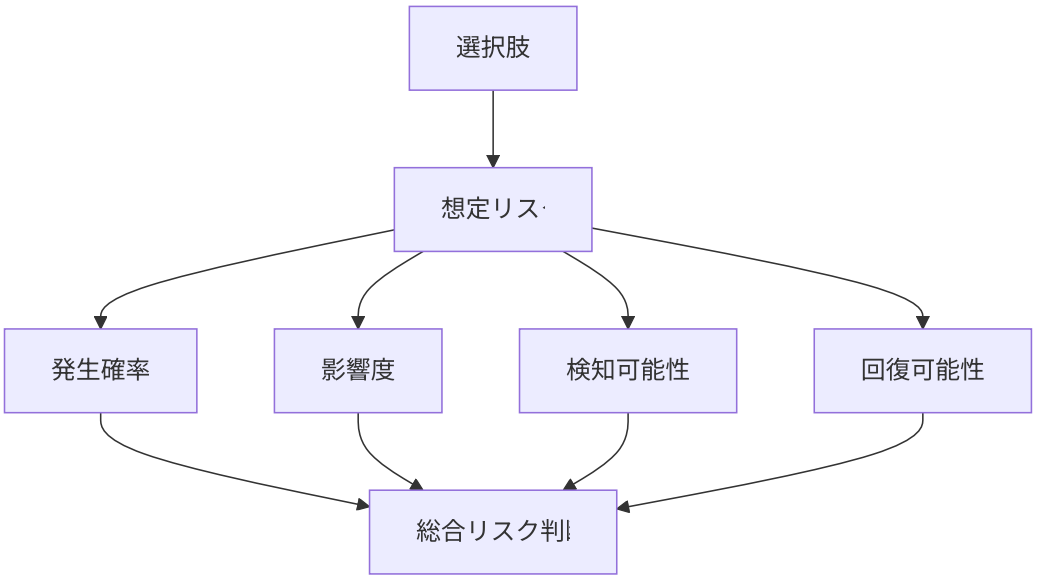
役割

- 過度な楽観を防ぐ
- 最悪ケースを先に想定する
- 予防策と緩和策を分ける
- 見えやすいリスク偏重を防ぐ
- 選択肢比較に安全性の観点を導入する

主な観点

- 発生確率
- 影響度
- 検知可能性
- 回復可能性
- 波及範囲
- 発生速度
- 隠れやすさ

基本構造



テンプレート

- 選択肢:
- 主なリスク:
- 発生確率:
- 影響度:
- 早期兆候:
- 検知方法:
- 予防策:
- 緩和策:
- 回復可能性:
- 最悪ケース:
- 総合判断:

典型リスク

- 実行不能
- コスト超過
- スケジュール遅延
- 品質低下
- 利害関係者反発
- 法制度不適合

- 運用定着失敗
- 一部成功・全体失敗
- 拡張不能
- 評判毀損

注意点

- 確率が低いだけで無視しない
- 致命傷リスクと軽微リスクを同列に扱わない
- 見えやすいリスクだけを重視しない
- 「何とかなる」で終わらせない

優先順位決定

優先順位決定

優先順位決定とは、複数の候補のうち、何を先に進め、何を後回しにし、何を捨てるかを決めることである。

良い案が複数あっても、資源が有限なら全て同時には進められない。
したがって、優先順位は「価値判断」だけでなく、「資源配分の設計」でもある。

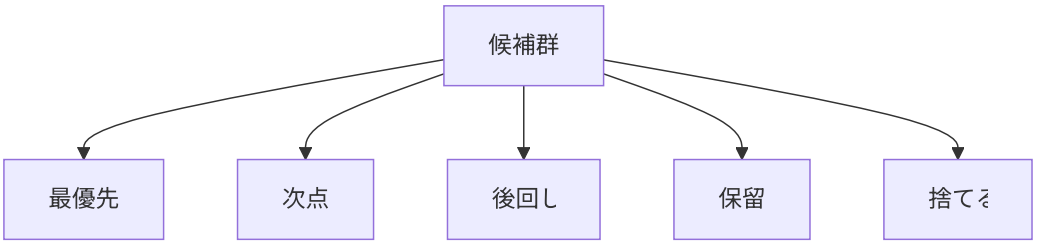
役割

- 実行順序を決める
- 資源集中先を決める
- 後回しにする理由を明示する
- 捨てる対象を決める
- 見直し条件を定める

何を見るか

- インパクト
- 緊急性
- 実行容易性
- 依存関係
- リスク低減効果
- 学習価値
- 可逆性
- 資源消費量

基本構造



テンプレート

- 候補一覧:
- 最優先:
- 次点:
- 後回し:
- 保留:
- 捨てるもの:
- 判断理由:
- 依存関係:
- 見直し条件:

典型判断軸

- 今やらないと損失が大きいか
- 先にやると後続が楽になるか
- 小さく始めて学べるか
- コストの割に効果が大きいか
- 致命的なリスクを減らせるか

注意点

- 緊急と重要を混同しない
- 声の大きい案件が上に来やすいことに注意する
- 「全部大事」を放置しない
- 優先順位は固定ではなく、状況依存で見直す

撤退基準設計

撤退基準設計

撤退基準設計とは、どの条件になったら、選んだ方針や施策を中止・縮小・変更するかを事前に決めておくことである。

撤退基準がないと、失敗している案に対しても、過去投入コストや感情的執着のために資源を流し続けやすい。
そのため、decision は採用判断だけでなく、やめる条件まで含めて設計されるべきである。

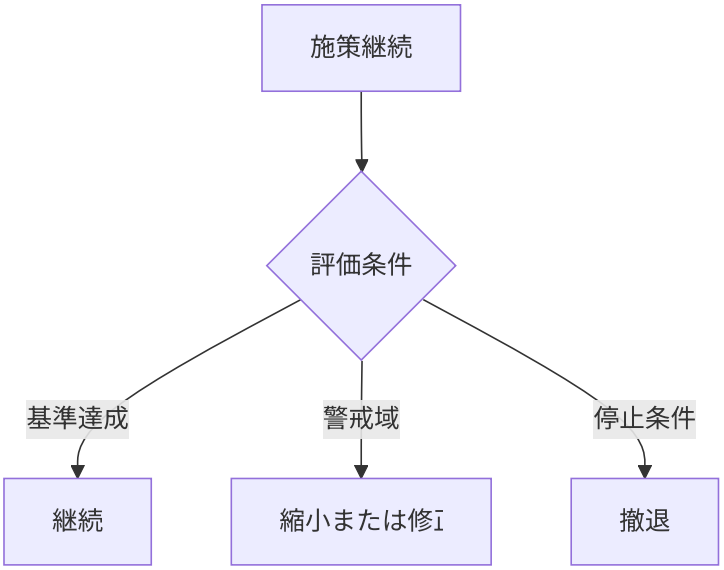
役割

- 損切りの遅れを防ぐ
- サンクコストへの執着を抑える
- 継続・縮小・停止の線を明確にする
- 感情判断ではなく条件判断にする
- 再挑戦条件を整える

何を見るか

- どの指標が閾値を下回ったらやめるか
- どの期間改善しなければ見直すか
- どのリスク発生で即停止か
- 部分撤退と全面撤退を分けるか
- 再挑戦の条件はあるか

基本構造



テンプレート

- 対象施策:
- 継続条件:
- 警戒条件:
- 撤退条件:
- 即時停止条件:
- 判定時期:
- 判定主体:
- 縮小案:
- 修正案:
- 再挑戦条件:
- 学びの記録方法:

典型的な撤退条件

- KPI が一定期間改善しない
- 想定コストを一定以上超過した
- 法的・安全上の重大リスクが顕在化した
- 運用主体が回らない
- 主要前提が崩れた
- 代替案が明確に優位になった

注意点

- 撤退を敗北とみなさない
- 一発退場条件と段階修正条件を分ける
- 基準を曖昧にすると結局やめられない
- 学びを回収しない撤退は損失が大きい

意思決定記録

意思決定記録

意思決定記録とは、何を、なぜ、その時点で採用または不採用にしたのかを残すことである。

記録がないと、後から結果だけを見て過去の判断を誤読しやすい。
また、判断の更新も、失敗からの学習も難しくなる。

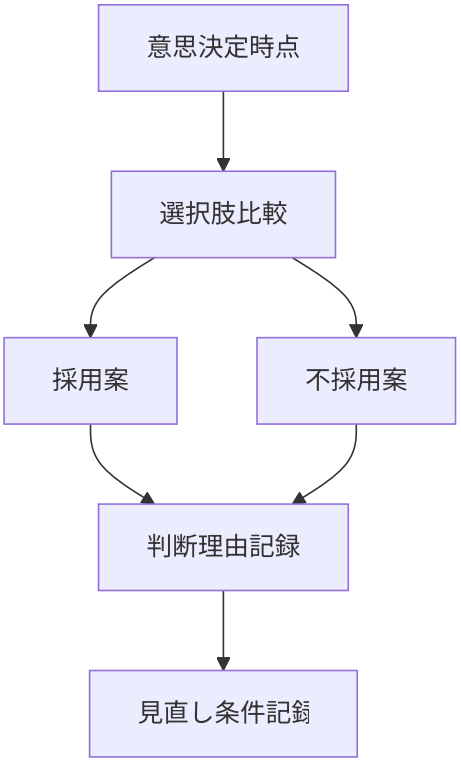
役割

- 判断過程を後から検証可能にする
- 結果論による改ざんを防ぐ
- 見直し時の起点を残す
- 共有判断の透明性を上げる
- 学習資産を蓄積する

記録すべき項目

- 何を決めたか
- いつ決めたか
- 何と比較したか
- なぜ採用したか
- なぜ不採用にしたか
- その時点の不確実性
- 想定リスク
- 見直し条件

基本構造



テンプレート

- 決定事項:
- 日時:
- 背景:
- 比較した選択肢:
- 採用理由:
- 不採用理由:
- 当時の不確実性:
- 想定リスク:
- 見直し条件:
- 次回確認日:
- 担当者:

注意点

- 事後合理化しない
- 結果が悪かったことと判断が悪かったことを混同しない

- 簡潔でよいので残さないことを避ける
- 当時見えていなかった情報を後から混入させない

要件定義

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

要件定義

要件定義とは、解決策が満たすべき条件を明確にすることである。

良い解決策は、先に「何を満たせばよいか」が定義されている。
逆に、要件が曖昧なまま作られた解決策は、あとから目的逸脱や運用破綻を起こしやすい。

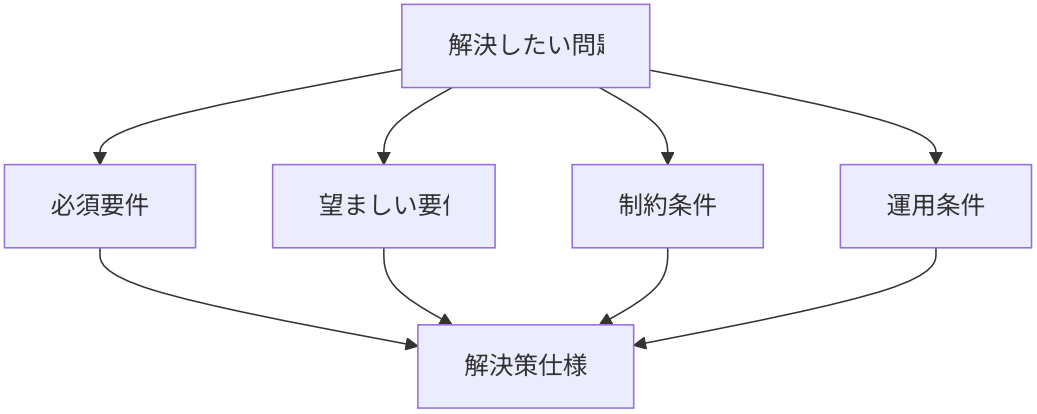
役割

- 解決策の成功条件を明文化する
- 必須条件と希望条件を分ける
- 制約条件を先に明らかにする
- 現場運用の前提を入れる
- 法制度や組織条件との衝突を避ける

要件の典型分類

- 機能要件
- 非機能要件
- 制約条件
- 法制度要件
- ユーザー要件
- 組織要件
- 運用要件
- 安全要件
- コスト要件

基本構造



テンプレート

- 解決したい問題:
- 想定利用者:
- 必須機能:
- 望ましい機能:
- 非機能要件:
- 制約:
- 法制度条件:
- 組織条件:
- 運用条件:
- 安全条件:
- 成功条件:
- 今回扱わない範囲:

注意点

- 要件と手段を混同しない
- 「便利そう」を要件と呼ばない
- 現場制約を書かずに理想設計にしない
- 後から足されそうな必須要件を先に掘り出す

Solution Design Hub

---

layer: hub
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

Solution Design Hub

solution_design は、decision で選ばれた方向性を、実際に動かせる解決策・制度・施策・運用へ変換する層である。

この層では、単なるアイデアではなく、「誰が」「何を」「どの順で」「何を使って」「どう測りながら」実行するかまで落とし込む。
したがって solution_design は、発想ではなく実装可能性を扱う。

役割

- 方針を実装可能な形に落とし込む
- 必要条件と要件を定義する
- どのレバーを動かせば結果が変わるかを定める
- 解決策を構造として組み立てる
- 実行プロセスを設計する
- 検証可能な形にする
- 失敗モードを事前に洗い出す
- 小規模試行から拡張までの道筋を作る

---

decision との違い

decision は「何を選ぶか」を扱う。
solution_design は「選んだものをどう動かすか」を扱う。

つまり、decision が方向を決め、solution_design が仕組みを作る。

下位ノート

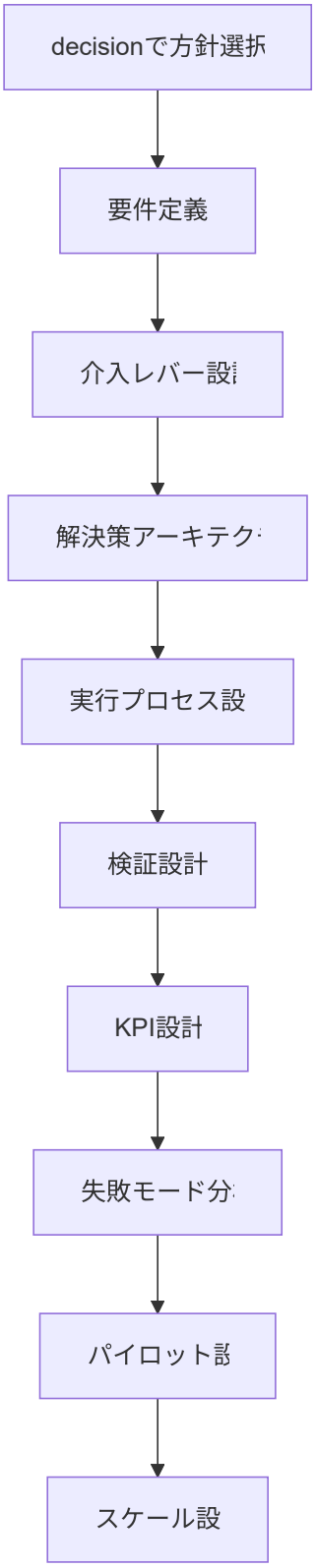
- 要件定義
- 介入レバー設計
- 解決策アーキテクチャ
- 実行プロセス設計
- 検証設計
- KPI設計
- 失敗モード分析
- パイロット設計
- スケール設計

基本原則

1. 解決策を思いつきで終わらせない
2. 要件と手段を混同しない
3. 仕組み・主体・フロー・測定をセットで設計する
4. 例外処理と失敗モードを先に見る
5. 小さく試し、学んでから広げる
6. 維持運用まで含めて考える

全体フロー

Desicion Hub



典型出力

- 解決策の仕様
- 実行フロー
- 担当主体
- 観測指標
- 検証方法
- 失敗時対応
- 小規模試行案
- 拡張条件

解決策アーキテクチャ

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

解決策アーキテクチャ

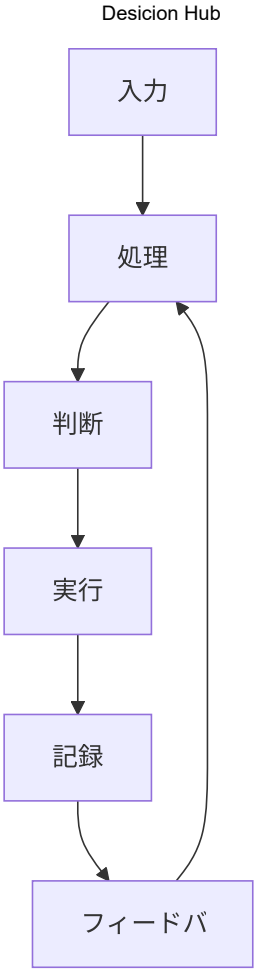
解決策アーキテクチャとは、複数の施策・主体・情報・判断・実行・記録を、一つの系として整合的に接続する設計である。

単発の施策は、単発の効果しか持たないことが多い。
一方、アーキテクチャとして設計された解決策は、入力からフィードバックまでを持ち、継続運用や改善が可能になる。

役割

- 解決策を部品の寄せ集めにしない
- 入力から改善までを一つの流れにする
- 主体と情報経路を明示する
- ボトルネックと脆弱点を見つける
- 改善可能な仕組みにする

基本構造



典型構成要素

- 入力
- 中核処理
- 判断点
- 実行主体
- 記録系
- フィードバック系
- 例外処理
- 代替経路

テンプレート

- 解決策名:

- 入力:
- 中核処理:
- 判断主体:
- 実行主体:
- 記録方法:
- フィードバック経路:
- 例外処理:
- 代替経路:
- ボトルネック:
- 壊れやすい箇所:

注意点

- 主体不明の工程を残さない
- フィードバックのない構造にしない
- 処理系と判断系を混ぜすぎない
- 一箇所停止で全体停止する設計に注意する

介入レバー設計

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

介入レバー設計

介入レバー設計とは、問題構造の中で、どの変数・行動・制度・情報・資源配分を動かせば結果が変わるかを定めることである。

原因を知ることと、変えられる点を知ることは別である。
このノートは「何が原因か」ではなく、「どこを動かすと現実が変わるか」に焦点を当てる。

役割

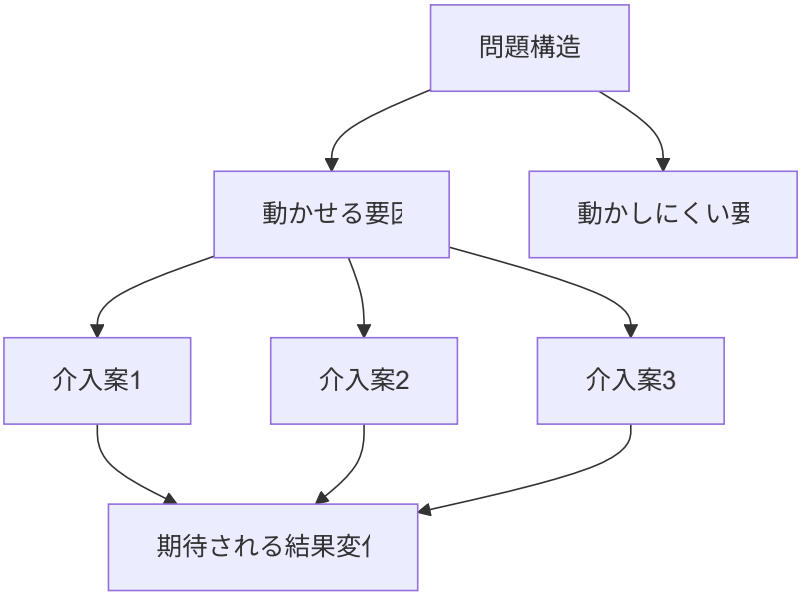
- 動かせる要因を特定する
- 効きやすい介入点を見つける
- 高インパクトと低コストの違いを整理する
- 副作用の大きい介入を見抜く
- 実装可能なレバーから優先順位をつける

レバーの典型類型

- 情報
- インセンティブ
- ルール
- 権限
- 配置
- フロー
- UI
- 教育
- 監視

- フィードバック頻度
- 資源配分
- 標準化

基本構造



テンプレート

- 目標結果:
- 現在の因果構造:
- 動かせる要因:
- 動かしにくい要因:
- 最小介入点:
- 高インパクト介入点:
- 実装しやすい介入点:
- 副作用:
- 優先介入案:
- 観察すべき変化:

注意点

- 動かせない原因ばかり追わない
- 効きそうでも実装不能なら設計として弱い
- 小さい介入でも連鎖的に効くことがある
- 介入レバーとKPIを結びつけて考える

実行プロセス設計

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

実行プロセス設計

実行プロセス設計とは、解決策を実際に回すために、誰が、いつ、何を、どの順で行うかを定義することである。

仕組みとして筋が通っていても、現場で回らなければ意味がない。
このノートでは、抽象的な構造ではなく、運用可能な手順に落とし込むことが中心となる。

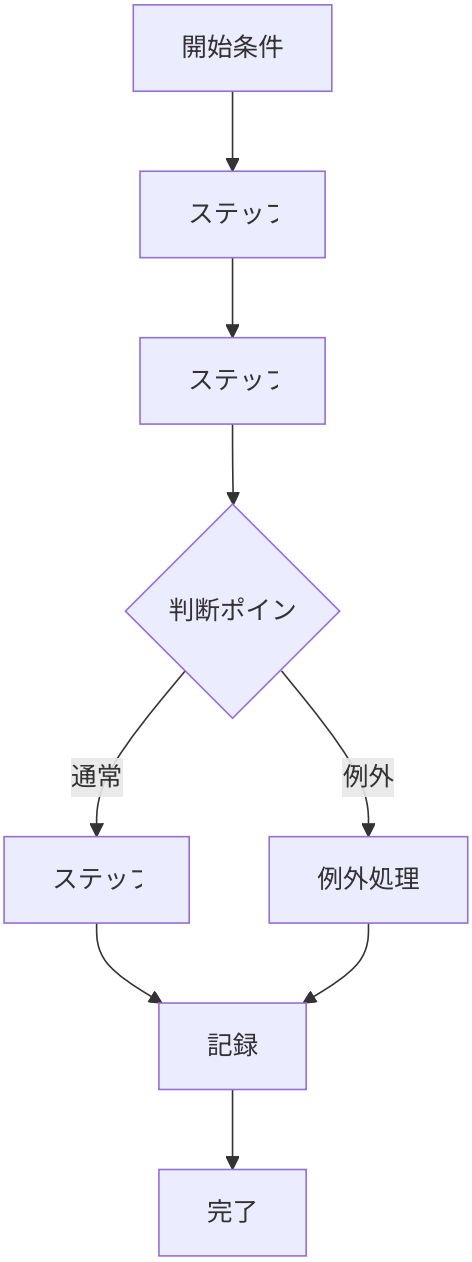
役割

- 手順を具体化する
- 担当主体を明確にする
- 判断ポイントを明示する
- 例外処理を織り込む
- 記録と受け渡しを設計する

主要観点

- 開始条件
- 担当者
- 実行順序
- 判断ポイント
- 例外処理
- エスカレーション
- 記録
- 完了条件

基本構造



テンプレート

- 開始条件:
- 担当者:
- ステップ1:

- ステップ2:
- ステップ3:
- 判断ポイント:
- 例外時対応:
- エスカレーション先:
- 記録項目:
- 完了条件:
- 次工程への受け渡し:

注意点

- 暗黙の前提に依存しない
- 例外時対応を書かないと現場で崩れる
- 善意や属人性に依存しすぎない
- 記録が後工程にどう使われるかを意識する

KPI設計

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

KPI設計

KPI設計とは、解決策が継続的に機能しているかを監視するための指標体系を作ることである。

重要なのは、結果指標だけでなく、先行指標、プロセス指標、安全指標、品質指標も合わせて持つことである。
結果だけを見ていると、問題が顕在化したときにはすでに手遅れになりやすい。

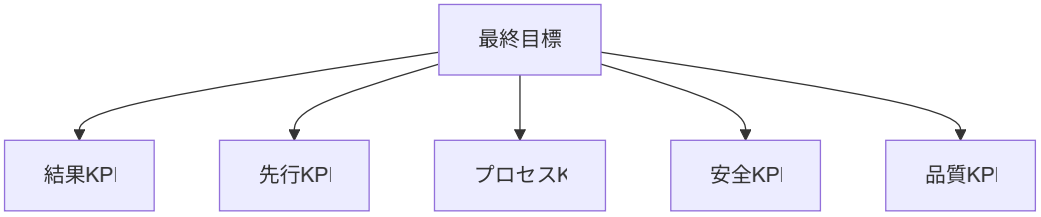
役割

- 継続監視の軸を作る
- 異常を早期に捉える
- 実行プロセスの詰まりを見つける
- 成果だけでなく健全性も監視する
- 撤退や拡張の判断材料を供給する

指標の層

- 結果指標
- 先行指標
- プロセス指標
- 安全指標
- 品質指標

基本構造



テンプレート

- 最終目標:
- 結果KPI:
- 先行KPI:
- プロセスKPI:
- 安全KPI:
- 品質KPI:
- 測定頻度:
- 取得方法:
- 責任者:
- 閾値:
- 見直し条件:

注意点

- 測りやすいものだけを追わない
- KPI のゲーム化に注意する
- 取得コストが高すぎる指標を増やしすぎない
- 一つの数値に全てを背負わせない

検証設計

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

検証設計

検証設計とは、解決策が本当に効いているかを判断するために、何をどう観察し、何と比較し、どの条件で成功または失敗とみなすかを定めることである。

導入しただけで満足しないための設計であり、solution_design の中でも特に学習と改善の起点になる。

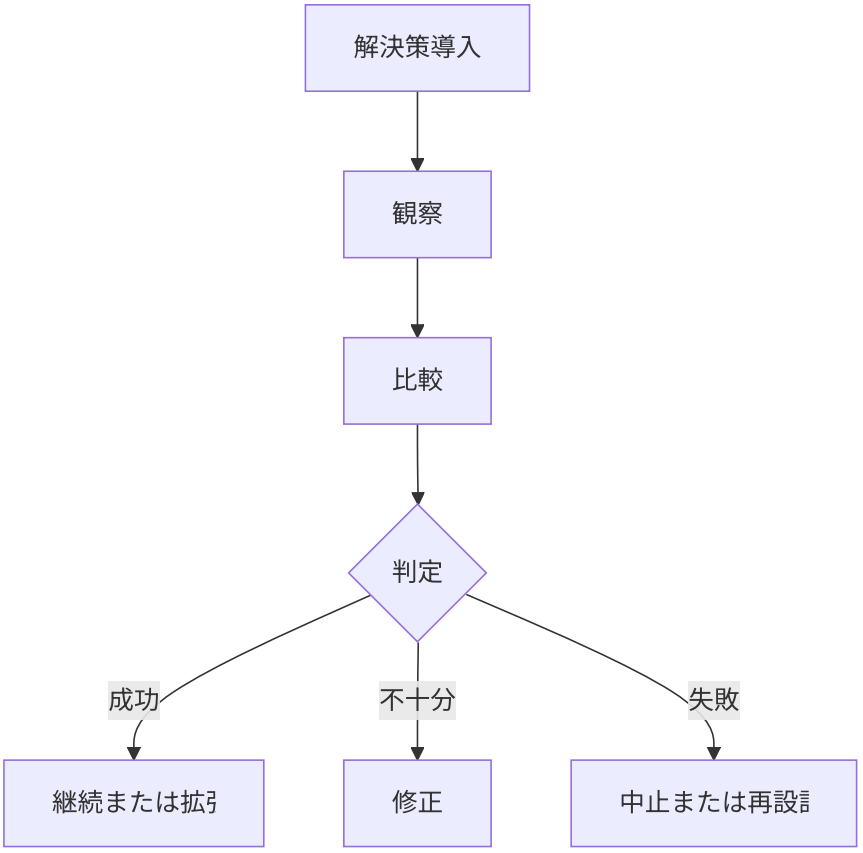
役割

- 成功の定義を明確にする
- 効果検証の方法を定める
- 比較対象を決める
- 副作用の観測を組み込む
- 継続・修正・中止の判断材料を作る

何を見るか

- 成功条件
- 比較対象
- 観察期間
- 効果発現ラグ
- 副作用
- 継続条件
- 中止条件

基本構造



テンプレート

- 対象施策:
- 検証目的:
- 主要指標:
- 比較対象:
- 観察期間:
- 期待変化:
- 想定副作用:
- 継続条件:
- 修正条件:
- 中止条件:
- 次に見るべき論点:

注意点

- 効果と副作用を分けて観る
- 導入直後のノイズで結論を急がない
- 比較対象なしに「効いた」と言い切らない
- 成功条件を後出しにしない

失敗モード分析

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

失敗モード分析

失敗モード分析とは、解決策がどのように壊れうるか、どの地点で失敗しやすいかを事前に洗い出すことである。

設計段階で失敗モードを想定しておくことで、脆弱点の補強、監視ポイントの明確化、例外処理の整備が可能になる。

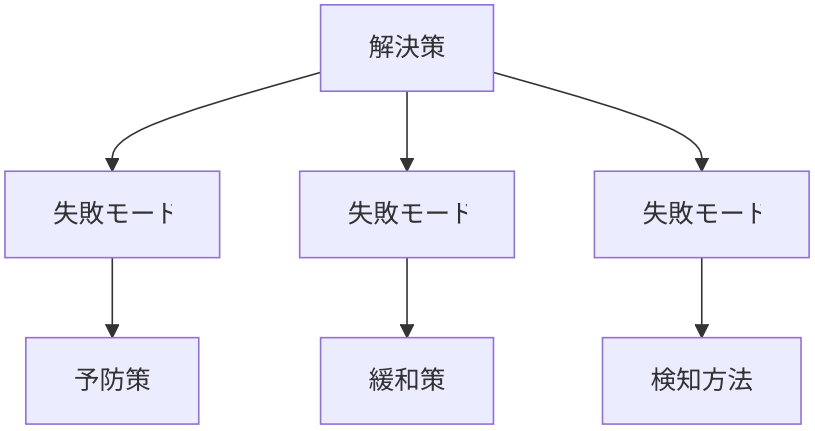
役割

- 想定外を減らす
- 脆弱点を先に見つける
- 人のミス以外の失敗も捉える
- 予防策と緩和策を分けて考える
- 実装前に設計品質を上げる

見るべき点

- どこで止まるか
- どこで誤作動するか
- 誰が詰まるか
- 何が誤解されるか
- 何が副作用になるか
- 何が隠れた損失になるか

基本構造



テンプレート

- 解決策:
- 想定失敗モード1:
- 想定失敗モード2:
- 想定失敗モード3:
- 発生原因:
- 影響:
- 検知方法:
- 予防策:
- 緩和策:
- 残余リスク:

注意点

- 人の注意不足だけに還元しない
- UI、情報、制度、運用、引継ぎの失敗も見る
- 検知不能な失敗を特に警戒する
- 稀でも致命的な失敗は重く扱う

スケール設計

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

スケール設計

スケール設計とは、パイロットで確認した解決策を、より広い対象・大きな規模・長い期間へ拡張する際に必要な条件を設計することである。

小さく回るものが、そのまま大きく回るとは限らない。
規模拡大では、人員、統制、品質、例外処理、教育、記録、インフラの問題が新たに発生しやすい。

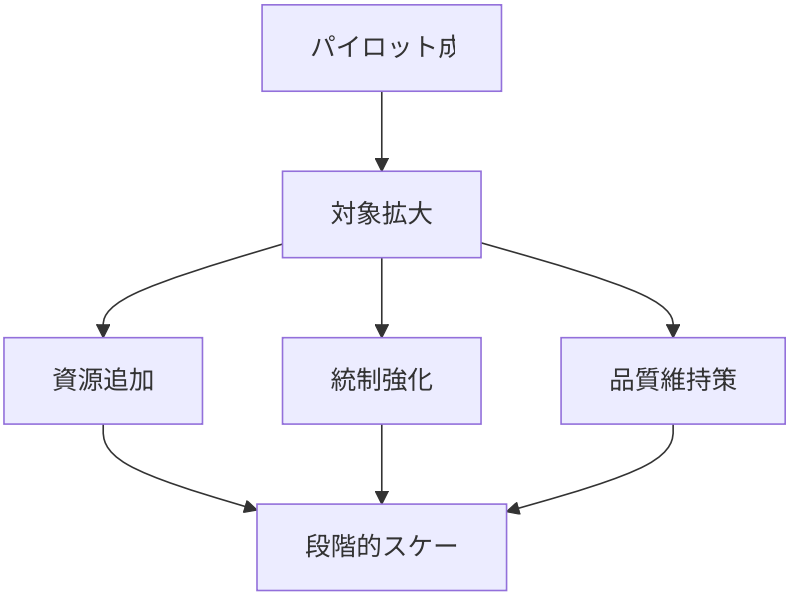
役割

- 拡張時の崩れ方を先に考える
- 必要資源の増加を見積もる
- 品質維持方法を設計する
- 統制と柔軟性のバランスを取る
- 段階的拡張の道筋を作る

何を見るか

- 何が増えるか
- 何が壊れやすくなるか
- 誰を追加する必要があるか
- 品質をどう保つか
- 例外処理をどう標準化するか
- どの条件で次段階へ進むか

基本構造



テンプレート

- 対象解決策:
- スケール対象:
- 増加要素:
- 必要資源:
- 追加主体:
- 品質維持策:
- 統制方法:
- 想定ボトルネック:
- スケール条件:
- 段階的拡張案:
- 拡張停止条件:

注意点

- パイロット成功をそのまま一般化しない
- 規模拡大で壊れる部分を先に見る
- 人と仕組みの両方を拡張する
- 例外処理を属人化させない

パイロット設計

---

layer: note
folder: thinking_engine/solution_design
status: stable
updated: 2026-03-14

パイロット設計

パイロット設計とは、解決策を小規模・限定条件下で試し、主要仮説や運用課題を確認するための設計である。

最初から全面展開せず、限定的に試すことで、失敗コストを抑えながら学習を得られる。
このノートは、単なる「お試し」ではなく、「何を学ぶために、どこまで試すか」を明確にする。

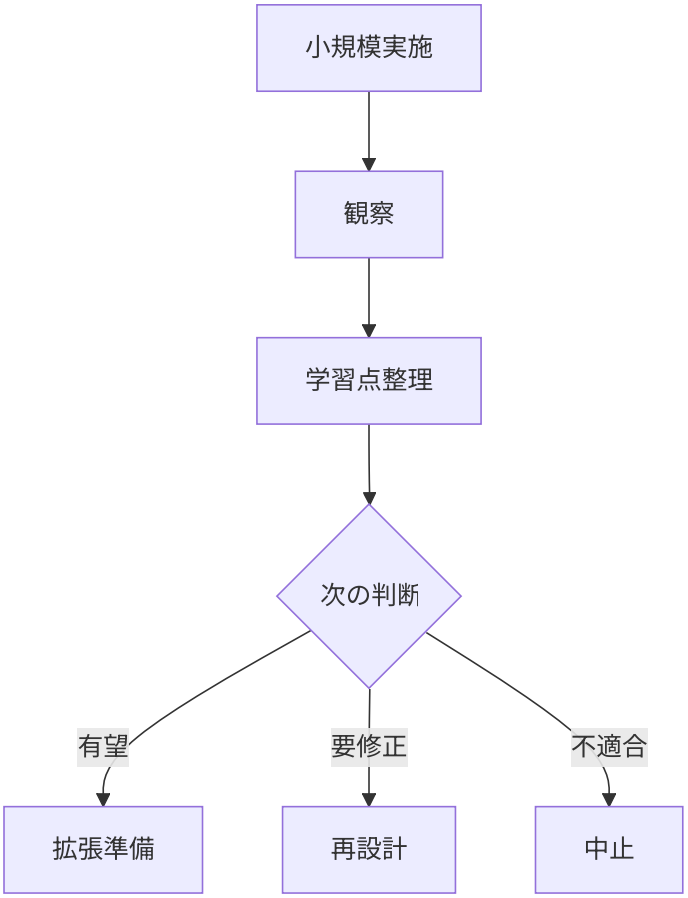
役割

- 小さく試して大きな失敗を避ける
- 実運用上の詰まりを早期に見つける
- 仮説検証と現場検証を両立する
- 本番展開前の修正材料を得る
- 次段階への条件を整える

何を見るか

- 何を試すか
- 何を学びたいか
- 対象範囲はどこまでか
- 成功条件は何か
- 中止条件は何か
- 本番との差は何か

基本構造



テンプレート

- 対象解決策:
- パイロットの目的:
- 対象範囲:
- 期間:
- 参加主体:
- 観察指標:
- 成功条件:
- 修正条件:
- 中止条件:
- 学びの記録:
- 次段階条件:

注意点

- 小さすぎて何も学べない試験にしない
- 本番条件と乖離しすぎない
- 見たい論点を増やしすぎない
- パイロット結果をそのまま一般化しない